3

KLASYFIKACJA

DRZEWA DECYZYJNE, KNN, NAIVE BAYES, MLP

KLASYFIKACJA I PODZIAŁ DATASETU

Na laboratoriach przetestujemy różne algorytmy klasyfikujące. Sprawdzimy czy uda się nam sklasyfikować irysy po gatunku, biorąc pod uwagę ich pomiary. Zobaczymy też czy lepiej klasyfikujemy ręcznie (my ludzie, zad 1) czy jednak AI (zadanie 2 i 3).

Zanim jednak przejdziemy do klasyfikacji, musimy wiedzieć jak dzielić zbiór na zbiór treningowy i testowy. Przejrzyj i uruchom poniższe wstawki pythonowe.

Korzystamy z załączonego iris_big.csv.

a) Użyj paczki pandas do wczytania tego pliku do Pythona i wyświetl bazę danych.

```
import pandas as pd

df = pd.read_csv("iris_big.csv")

print(df)
```

b) Pora na podział zbioru na zbiór testowy i treningowy. Wykorzystamy do tego funkcję train_test_split z paczki sklearn. Dodaj na górze programu odpowiedni import:

```
from sklearn.model_selection import train_test_split
```

Następnie dopisz na dole programu funckję dzielącą zbiór df.values na dwa kawałki w proporcjach 70% i 30%, w losowy sposób.

```
#podzial na zbior testowy (30%) i treningowy (70%), ziarno losowosci = 13
(train_set, test_set) = train_test_split(df.values, train_size=0.7,
random_state=13)
```

c) Sprawdź jakie rekordy znalazły się w zbiorze testowym i ile ma on rekordów.

```
print(test_set)
print(test_set.shape[0])
```

d) Bywa, że zbiór testowy i treningowy trzeba podzielić dodatkowo na inputy (kolumny numeryczne) i klasy (odmiany irysów). Wówczas z 2 zbiorów danych zrobią się nam cztery.

```
train_inputs = train_set[:, 0:4]
train_classes = train_set[:, 4]
test_inputs = test_set[:, 0:4]
test_classes = test_set[:, 4]
```

Sprawdź zawartość czterech zbiorów.

Grzegorz Madejski
grzegorz.madejski@ug.edu.pl
Zakład Sztucznej Inteligencji,
Instytut Informatyki, Uniwersytet Gdański

💻 ZADANIE 1: **KLASYFIKACJA PRZEZ CZŁOWIEKA**

Naszym zadaniem jest napisanie prostego klasyfikatora dla bazy irysów i zewaluowanie go. Klasyfikator skonstruujemy i zewaluujemy ręcznie, bez użycia AI. W zadaniu 2 skorzystamy z kolei z metod AI i porównamy czy lepszy wynik mają ludzie (Ty) czy maszyny.

Rozpocznij od przygotowania zbioru treningowego i testowego, tak jak w poprzednim zadaniu.

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("iris_big.csv")

(train_set, test_set) = train_test_split(df.values, train_size=0.7,
random_state=13)
```

Random state ustaw na numer swojego indeksu (zamień 13 na inny numer).

a) Napiszemy w Pythonie funkcję, która na podstawie czterech numerycznych parametrów irysa odgadnie jego gatunek wykorzystując do tego jedynie dwie instrukcje if, elif oraz else (jednokrotnie).

```
def classify_iris(sl, sw, pl, pw):
    if sl > 4:
        return("setosa")
    elif pl <= 5:
        return("virginica")
    else:
        return("versicolor")
```

Widać, że funkcja wykorzystuje sepal.length i petal.length, sprawdzając ich wartości. Na tej podstawie funkcja decyduje, jaką odmianę ma irys. Funkcja ta działa dla jednego irysa.

b) Chcemy teraz tę funkcję uruchomić dla wszystkich irysów ze zbioru testowego i zliczyć ile razy dobrze zgadnie odmianę irysa. Zrobimy to w dość prostu sposób – wykorzystując pętlę for. Schemat działania:

Dla każdego irysa ze zbioru testowego: sprawdź czy odpowiedź, którą zwraca funkcja classify_iris, jest taka sama jak prawdziwa odmiana irysa z ostatniej kolumny. Jeśli tak, to dodaj ten rekord do licznika dobrych odpowiedzi. Na końcu podaj ile rekordów funkcja classify_iris zgadła (liczbowo i procentowo).

Przełóżmy to na kod Pythonowy. Uwaga! Poniższy kod ma dwa miejsca ze znakami zapytania, które trzeba uzupełnić odpowiednimi fragmentami kodu!

```
good_predictions = 0
len = test_set.shape[0]

for i in range(len):
    if classify_iris(????) == test_set[????]:
        good_predictions = good_predictions + 1

print(good_predictions)
print(good_predictions/len*100, "%")
```

Jaka odpowiedź wyszła? U mnie niestety kiepska ☹️ 14 zgadniętych irysów (na 45 możliwych), co dało około 31%.

Grzegorz Madejski
grzegorz.madejski@ug.edu.pl
Zakład Sztucznej Inteligencji,
Instytut Informatyki, Uniwersytet Gdański

c) Pora na poprawienie funkcji classify_iris, tak aby działała lepiej (Kto kupi nasz program, jeśli działa beznadziejnie?). Wyświetl zbiór treningowy np. poprzez...

```
print(train_set)
```

(możesz dla ułatwienia też go posortować wg gatunku – jak to zrobić?)
...i przyglądając się mu, postaraj się znaleźć jakieś fajne zależności między pomiarami a odmianami irysa (użyj jedynie oczu i mózgu! 😉)
Nie zaglądaj do zbioru testowego!

Twoim zadaniem jest poprawić dwa warunki w ifach w funkcji classify iris, tak aby ta funkcja działała dobrze. Reszty funkcji nie modyfikuj.

```
def classify_iris(sl, sw, pl, pw):
    if ?????:
        return("setosa")
    elif ?????:
        return("virginica")
    else:
        return("versicolor")
```

Zadanie możesz uznać za zakończone, gdy procent poprawnie odgadniętych odpowiedzi, który nazywamy dokładnością klasyfikatora (ang. accuracy), będzie wynosił więcej niż 80%.

Czy uda Ci się przekroczyć 90%?

💻 ZADANIE 2: **DRZEWA DECYZYJNE**

W poprzednim zadaniu, w funkcji classify_iris, stworzyliśmy małe binarne drzewo decyzyjne postaci:

Musieliśmy jednak stworzyć to drzewo sami. Są jednak algorytmy, takie jak ID3 czy C4.5, które tworzą takie drzewa automatycznie i to z o wiele większą precyzją niż człowiek. W Pythonie można skorzystać z paczki sklearn (tree). Należy skorzystać ze źródeł internetowych, aby rozwiązać to zadanie, np.

- https://scikit-learn.org/stable/modules/tree.html, lub
- https://medium.com/@haydar_ai/learning-data-science-day-21-decision-tree-on-iris-dataset-267f3219a7fa

Pamiętaj, że korzystamy ze zbioru iris_big.csv z 1500 irysami.
Wykorzystując wiedzę z samouczków i poprzednich zadań wykonaj następujące polecenia.

a) Podziel w losowy sposób bazę danych irysów na zbiór treningowy i zbiór testowy w proporcjach 70%/30%. Wyświetl oba zbiory. Podziel te zbiory na cztery części (inputy i class), jeśli jest taka potrzeba.
b) Zainicjuj drzewo decyzyjne metodą DecisionTreeClassifier.
c) Wytrenuj drzewo decyzyjne na zbiorze treningowym, wykorzystując funkcję fit.

Grzegorz Madejski
grzegorz.madejski@ug.edu.pl
Zakład Sztucznej Inteligencji,
Instytut Informatyki, Uniwersytet Gdański

d) Wyświetl drzewo w formie tekstowej i/lub w formie graficznej. Jeśli masz problemy z tym podpunktem – pomiń go.
e) Dokonaj ewaluacji klasyfikatora: sprawdź jak drzewo poradzi sobie z rekordami ze zbioru testowego. Wyświetl dokładność klasyfikatora, czyli procent poprawnych odpowiedzi. Wykorzystaj do tego funkcję score lub predict.
f) Wyświetl macierz błędów (ang. confusion matrix), która zestawia liczby błędnych i poprawnych odpowiedzi, dla wszystkich klas.

Czy drzewo nauczone automatycznie jest lepsze niż Twoje drzewko stworzone ręcznie w poprzednim zadaniu? 😉 Kto wygrał Ty czy AI?

💻 ZADANIE 3: **INNE KLASYFIKATORY**

Dla zbioru danych z irysami przeprowadź klasyfikację metodą k-najbliższych sąsiadów dla kilku przypadków:

- k-NN, k=3
- k-NN, k=5
- k-NN, k=11

oraz klasyfikatorem Naive Bayes, oraz prostą siecią neuronową (Multilayer Perceptron = MLP)

W rozwiązaniu zadania uwzględnij następujące punkty:

a) Podziel w losowy sposób bazę danych na zbiór treningowy (70%) i testowy (30%).
b) Uruchom każdy z klasyfikatorów wykorzystując paczki i dokonaj ewaluacji ma zbiorze testowym wyświetlając procentową dokładność i macierze błędu (cofusion matrix). Wygooglaj "Sklearn scikit" + nazwa klasyfikatora, żeby znaleźć przydatne materiały. Warto też zajrzeć tutaj: https://scikit-learn.org/stable/supervised_learning.html
c) Porównaj wszystkie 5 klasyfikatorów (DecTree z zad 2, 3NN, 5NN, 11NN, NB, MLP) pod względem dokładności. Który z nich jest najlepszy?

💻 ZADANIE 4: **DIAGNOZA CHOROBY I MIARY OCENY**

Pobierz teraz załączony plik diagnosis.csv, gdzie na podstawie trzech parametrów numerycznych diagnozowana jest choroba (0 = test negatywy, zdrowy, 1 = test pozytywny, chory).

Zasadniczo należy powtórzyć eksperyment z poprzedniego zadania (porównanie wydajności klasyfikatorów). Dodajmy jednak kilka wymagań:

a) Przed klasyfikacją zrób trójwymiarowy punktowy wykres danych (parametry medyczne, a na legendzie osoby chore na czerwono i zdrowe na niebiesko). Możesz zrobić nawet rotujący wykres GIF z pomocą AI jeśli chcesz.
b) Dla każdego z klasyfikatorów policz: Accuracy, Precision, Recall/Sensitivity, i podaj graficzną tabelkę błędów z wykorzystaniem paczki Seaborn.
c) Odpowiedz na pytanie (np. w komentarzu do zadania) jak definiujmy i jak rozumiem trzy wyżej wymienione miary? Która z ich jest ważna jeśli nie chcemy klasyfikować osób zdrowych jako chore? A która jest ważna by minimalizować ryzyko pomyłki, że osobę chorą klasyfikujemy jako zdrową?
d) Odpowiedz na pytanie: jeśli zbiór jest niezbalansowany (dużo osób zdrowych, mało chorych), to czy Accuracy jest bezpieczną miarą?

Grzegorz Madejski
grzegorz.madejski@ug.edu.pl
Zakład Sztucznej Inteligencji,
Instytut Informatyki, Uniwersytet Gdański